

DriveAnchor: Progressive Anchor-based Flow Learning for Autonomous Driving Planning

Limin Yan^{*,†}

Meituan Autonomous Driving
yanlimin03@meituan.com

Haoyun Tang^{*}

Meituan Autonomous Driving
Xi'an Jiaotong University
haoyuntang224@163.com

Yutao Qiu

Meituan Autonomous Driving
Beijing Institute of Technology
3120240873@bit.edu.cn

Hongqing Liu

Meituan Autonomous Driving
JaimeThomas01@outlook.com

Haoyu Xu[§]

Meituan Autonomous Driving
xuhaoyu06@meituan.com

Abstract: We present **DriveAnchor**, a three-stage framework for autonomous driving planning that achieves behavioral diversity, controllability, and safety in a composable pipeline. *Demonstration Flow Pretraining* replaces the unstructured Gaussian prior with a vocabulary of 2,398 trajectory shapes constructed by farthest-point sampling, structurally grounding behavioral diversity in vocabulary coverage. *Guided Flow Post-training* jointly post-trains an Energy Field module with FM—conditioning EF on static road geometry alone—to relocate anchors toward user-specified corridor polygons before flow generation, adding controllability without differentiable guidance; after Stage 2, new corridor presets require only EF updates, not FM retraining. *Reward-Refined Flow Fine-tuning* applies zeroth-order RL to align each anchor’s output with collision-avoidance objectives: because f_θ is a deterministic feedforward network in single-step mode, each anchor uniquely determines the output trajectory, reducing reward optimization to a direction search in anchor space without $\log \pi_\theta$ computation or ODE-to-SDE conversion. Evaluated on approximately 2 million held-out driving scenarios, DriveAnchor reduces near-range collision rates by 89% and improves mean reward by 32% without degradation in imitation accuracy, with 2.06 ms inference on NVIDIA Drive Orin. DriveAnchor has been validated through real-world vehicle testing (Figure 4), confirming its practicality for production deployment.

Keywords: Autonomous Driving, Flow Matching, Reinforcement Learning, Anchor-based Planning

1 Introduction

Flow matching [1, 2] has emerged as a strong generative backbone for autonomous driving planning, offering deterministic, efficient, one-step inference. Yet deploying these models in safety-critical settings exposes three gaps: (i) *no structural diversity guarantee*—without a structured prior, rare maneuvers silently collapse at test time; (ii) *no real-time controllability*—a deployed FM planner cannot be steered toward a target corridor without expensive retraining; (iii) *imitation-only training cannot optimize safety*—supervised FM reproduces training distribution tail behaviors [3, 4] and cannot directly optimize collision avoidance. Together, these gaps prevent a single generative model from being production-ready across the full operational spectrum of driving demands.

Addressing all three gaps simultaneously proves difficult. Existing controllability mechanisms [5, 6] require either gradient-compatible continuous rewards or full FM retraining whenever corridor specs

^{*}Equal contribution. [†]Lead corresponding author. [§]Co-corresponding author.

change—both impractical at production deployment. Safety alignment via RL is further blocked: standard policy gradient requires $\log \pi_\theta(x)$, which is analytically intractable for flow matching’s deterministic ODE [7], and driving rewards are non-differentiable (per-scene collision detection), ruling out the differentiable reward models assumed by prior RL fine-tuning work [8, 9, 10]. No prior method addresses all three gaps without significant model approximation or retraining overhead.

DriveAnchor makes three contributions:

A composable three-stage framework for production deployment. Demonstration Flow (diversity), Guided Flow (controllability), and Reward-Refined Flow (safety) are fully decoupled: each stage adds one property without disrupting the others, and each is independently updatable—new corridor presets require only retraining EF; updated safety objectives require only rerunning Stage 3, without touching the rest of the pipeline.

A geometry-driven controllability module trained independently of the reward signal. The Energy Field (EF) operates *before* flow generation, relocating anchors toward user-specified corridor polygons using only static road geometry—no differentiable guidance, no scene-specific reward required. Different driving behaviors (lane keep, turns, lane changes) are encoded by varying the polygon and exit edge, making EF a reusable, preset-based layer that leaves FM and Stage 3 intact.

Zeroth-order RL fine-tuning of flow matching without log-likelihood. Since f_θ is deterministic in single-step mode, anchor x_0 uniquely determines the output trajectory. The FPS-constructed vocabulary provides a kinematically feasible, angularly diverse finite-difference basis in trajectory space—enabling reward optimization as a direction search in anchor space with no $\log \pi_\theta$ computation, no ODE-to-SDE conversion, and no differentiable reward signal required—applicable to any black-box safety oracle, including hand-crafted collision rules and learned safety classifiers.

2 Related Work

Generative planning and structured priors. Diffusion models [11, 12, 13, 14, 15, 16] and flow matching [1, 17, 18, 2] have advanced trajectory generation in autonomous driving and robotics—for example, Janner et al. [13] showed that diffusion models can plan flexibly over long horizons, and Tan et al. [2] applied flow matching to achieve state-of-the-art results on nuPlan. End-to-end planners [19, 20, 21, 22, 23] also achieve strong results—UniAD [19] unified perception and planning in a single transformer—but all remain purely imitative. Scene encoding via vectorized representations [24, 25] and multi-sensor fusion [26] provides the perceptual backbone for these planners. Prior anchor-based methods [27, 28, 29, 30, 31, 32, 33, 34, 35, 36] use anchors as *scoring primitives* for post-hoc mode selection—e.g., MTR [32] combines intention-based anchors with a transformer for joint multi-agent prediction—not as generative priors. DriveAnchor instead uses FPS-constructed anchors as structured initial conditions for the flow ODE, structurally grounding behavioral diversity in vocabulary coverage.

Controllable generation. Ho and Salimans [5] proposed classifier-free guidance for diffusion models by interpolating conditional and unconditional score estimates at inference. Zhong et al. [37] extended this to driving by injecting differentiable cost signals into the diffusion sampling process to steer traffic trajectories. More broadly, differentiable cost injection [6, 38] and in-training conditioning require either gradient-compatible rewards or full model retraining whenever corridor specs change—both impractical at deployment. DriveAnchor’s Energy Field is post-trained jointly with FM as a separate stage, operates on static road geometry, and requires no differentiable signal.

RL fine-tuning of generative models. Ouyang et al. [39] showed that reward-aligned fine-tuning via RLHF substantially improves language model behavior; Shao et al. [40] further scaled this with group-relative policy optimization [40], establishing the paradigm of optimizing non-differentiable rewards over discrete outputs. Black et al. [8] adapted this to diffusion models by framing denoising as an MDP, but as with ReinFlow [9] and DanceGRPO [10], all of these methods require $\log \pi_\theta(x)$, intractable for flow matching’s deterministic ODE [7]. Offline sequence modeling [41] and robot control via Diffusion Policy [42] sidestep policy gradients but cannot optimize non-differentiable rewards. Malladi et al. [43] showed that ZO can fine-tune billion-parameter LLMs at inference-level memory overhead; DriveAnchor instead applies ZO directly in *output trajectory space*, exploiting anchor identity to naturally avoid distributional assumptions and ODE approximations.

3 Method

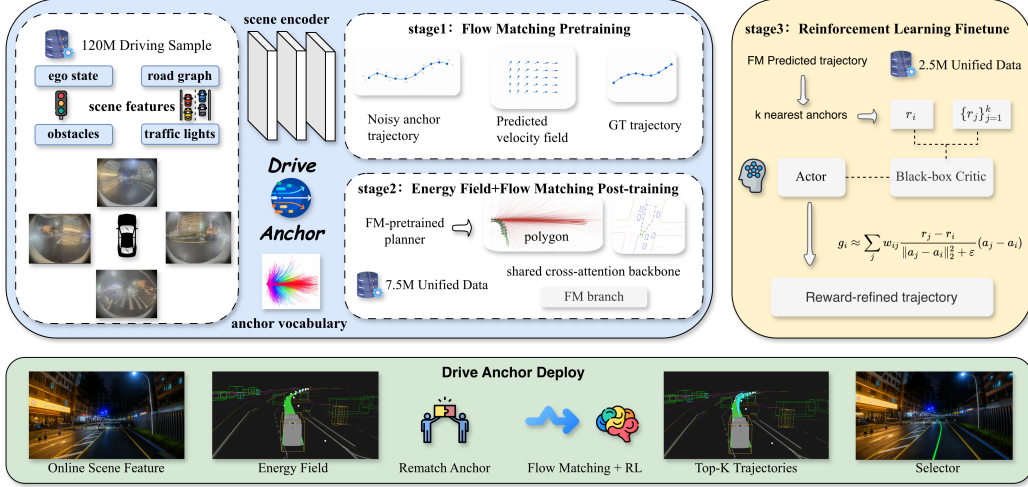


Figure 1: Overview of the DriveAnchor framework. The three-stage pipeline progresses from **Stage 1** (FM Pretraining for diversity) through **Stage 2** (EF post-training for controllability) to **Stage 3** (RL fine-tuning for safety). At inference, EF and FM together generate a diverse trajectory set, from which a downstream cost selector retains the top- K candidates. See Sections 3.2–3.4 for details.

Figure 1 illustrates the full DriveAnchor pipeline. The framework is composed of three decoupled stages. **Stage 1** (Section 3.2) pretrains the Flow Matching network f_θ with a structured anchor prior—replacing an unstructured Gaussian with a vocabulary of 2,398 FPS-constructed trajectory shapes—to ground behavioral diversity in vocabulary coverage. **Stage 2** (Section 3.3) jointly post-trains an Energy Field (EF) module that conditions on static road geometry to relocate anchors toward a user-specified corridor before generation, adding run-time controllability without FM retraining. **Stage 3** (Section 3.4) applies zeroth-order RL to fine-tune f_θ for collision avoidance: because f_θ is deterministic in single-step mode, each anchor uniquely determines the output trajectory, reducing reward optimization to a direction search in anchor space. At inference, the scene encoder first encodes context; EF then proposes a corridor-guided anchor displacement and rematches it to the nearest vocabulary anchor to keep the decoder input in distribution; FM then generates one trajectory per anchor, and top- K candidates are passed to a downstream cost selector.

3.1 Preliminaries: Flow Matching for Planning

Flow matching [1, 44, 17, 45] learns a velocity field $v_\theta(x_t, t)$ that transports a prior p_0 to an expert distribution p_1 along straight linear paths by minimizing:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, x_0, x_1} \left[\left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2 \right] \quad (1)$$

where $t \sim \text{Uniform}(0, 1)$ and $x_t = (1 - t)x_0 + tx_1$, offering more efficient inference than diffusion models [11, 46]. At inference, integrating v_θ from $t=0$ to $t=1$ is *deterministic*; we denote the resulting map as $f_\theta(x_0, s)$: the same starting point x_0 and scene context s always yield the same output $x_1 = f_\theta(x_0, s)$ —a property Stage 3 exploits for its zeroth-order direction search.

DriveAnchor’s key departures. Rather than sampling x_0 from an unstructured Gaussian prior, DriveAnchor uses a *structured anchor prior* $x_0 \in \mathcal{A}$: a vocabulary of 2,398 kinematically feasible trajectory shapes covering distinct maneuver classes, built by FPS over 100M+ real-world frames. Since every GT trajectory lies close to some anchor, the residual $\Delta x = x_1 - x_0$ is small, enabling a *single-step approximation* $\hat{x}_1 = f_\theta(x_0, s)$ with $\mathcal{O}(|\Delta x|^2)$ error—analogous to the single-step inference of consistency models [47]—and empirically, two sequential passes (denoted FM*2; notation defined in Section 4) further reduce min-ADE. The model is also trained without explicit t input, as described in Eq. (3).

3.2 Demonstration Flow Pretraining

The FM independently learns $p(\text{GT} \mid \text{anchor})$ for each of $M=2,398$ anchors built by iterative farthest-point sampling [48] over a historical corpus $\mathcal{H}$ of 100M+ frames:

$$\mathcal{A} = \text{FPS}\left(\{a_k^*\}_{k=1}^{|\mathcal{H}|}, M\right), \quad a_k^* \in \mathbb{R}^{T \times 2} \quad (2)$$

The corpus is temporally disjoint from training and evaluation data. During training, for each scene a single anchor is randomly sampled from $\mathcal{A}$ and the model is trained on a noise-interpolated input:

$$x_t = (1 - \alpha) \tilde{x}_0 + \alpha \text{GT}, \quad \alpha \sim \text{Uniform}(0, 1) \quad (3)$$

where α plays the role of t in Eq. (1) with $\tilde{x}_0$ as the structured anchor prior (replacing the Gaussian x_0); the model predicts $\Delta x = \text{GT} - x_t$ without knowing α , and at inference $\alpha=0$ so $f_\theta(x_0, s)$ directly recovers $\text{GT} - x_0$. At inference, the full vocabulary of M anchors is evaluated simultaneously in a single forward pass, producing M candidate trajectories. The anchor table additionally serves as the reference set for Stage 3 ZO gradient estimation, where its FPS-uniform coverage ensures diverse finite-difference directions across all maneuver classes.

Anchor density. $M=2,398$ anchors cover diverse driving behaviors with uniform spatial coverage (Figure 1), keeping GT trajectories close to the vocabulary and improving FM mode precision. The same density also benefits Stage 3 by providing usable finite-difference directions; with $M \approx 250$, RL made little progress due to poor 160-D coverage.

Retrieval with ϵ -ball constraint. Naive nearest-neighbor retrieval can cause *mode collapse* in sparse anchor regions: a generated trajectory with no close same-mode neighbors may draw anchors from structurally unrelated maneuver types, conflating gradient signals across modes (see Appendix G.2, Figure 7). We address this by combining KNN with an ϵ -ball filter: a candidate a_j is admitted only if it is among the N nearest by distance *and* satisfies $\|a_i - a_j\|_2 \leq \epsilon$, where ϵ is the per-cluster median inter-anchor distance. This keeps the effective neighborhood within a single maneuver mode regardless of local density. We define the neighborhood of a_i as:

$$\mathcal{N}(a_i) = \left\{ a_j \in \mathcal{A} \mid \|a_i - a_j\|_2 \leq \epsilon \wedge a_j \in N \text{ nearest of } \mathcal{A} \right\} \quad (4)$$

We use a pre-built KD-tree index over $\mathcal{A}$ for $\mathcal{O}(\log M)$ lookup, enabling sub-millisecond neighborhood retrieval during Stage 3 training ($N=16$ in all experiments; see ablation in Section 4.3).

3.3 Guided Flow Post-training

Stage 1 provides no controllability mechanism. The **Energy Field (EF)** fills this gap before Stage 3: a learned module that predicts a residual displacement, jointly post-trained with FM (with $\mathcal{L}_{\text{FM}}$ as regularizer and encoder frozen), taking the scene encoding, a corridor polygon query, and the current anchor as input and outputting $\Delta v_{\text{EF}} \in \mathbb{R}^{T \times 2}$ to relocate anchors before generation. A polygon specifies the *region* the trajectory must traverse and the exit *edge*, encoding different maneuvers (lane keep, turns, lane changes) without retraining the base FM or re-running Stage 3.

At inference, EF and FM are applied as a two-step pipeline:

$$\tilde{a}_{\text{EF}} = a_{\text{anchor}} + \Delta v_{\text{EF}}, \quad a_{\text{rematch}} = \arg \min_{a \in \mathcal{A}} \|\tilde{a}_{\text{EF}} - a\|_2 \quad (5)$$

$$a_{\text{final}} = f_\theta(a_{\text{rematch}}, s) \quad (6)$$

where $a_{\text{anchor}} \in \mathcal{A}$ is the selected anchor, Δv_{EF} is the EF displacement correction, and s is the scene context. EF produces a continuous corridor-guided proposal, which is projected back to the nearest vocabulary anchor before FM decoding to preserve in-distribution inference and kinematic quality. The resulting two-step process can be applied iteratively, with each FM output serving as the next anchor.

Design rationale. EF and FM operate on orthogonal signals (static geometry vs. dynamic agents); keeping them separate avoids FM retraining on every preset update. Conceptually, $\|\Delta v_{\text{EF}}\| \approx 0$ means the anchor is already inside the corridor [49].

Training objective. A trajectory $a = (a_1, \dots, a_T)$ is *good* if it enters the target polygon $\mathcal{P}$ and exits only through the designated edge $\mathcal{E}_k$:

$$\text{good}(a) = \mathbb{K}[\exists t : a_t \in \mathcal{P}] \wedge \mathbb{K}[\text{no exit through } \partial\mathcal{P} \setminus \mathcal{E}_k] \quad (7)$$

EF is trained with a regression loss where the target displacement is:

$$\delta^*(a) = \begin{cases} \mathbf{0} & \text{if good}(a) \\ a_{\text{near-good}} - a_{\text{noisy}} & \text{otherwise} \end{cases} \quad (8)$$

where a_{noisy} is the noise-perturbed anchor input and $a_{\text{near-good}}$ is the nearest good trajectory from the clean vocabulary. Good trajectories are preserved (zero correction); bad trajectories are pulled toward the nearest good trajectory. Classification is purely geometric and scene-obstacle-independent, so EF provides no collision-avoidance guarantee. The exit edge $\mathcal{E}_k$ encodes the target maneuver—switching only this parameter selects straight driving, turns, or lane changes without retraining; a passable-segment variant additionally handles lane changes with a hard-end ahead. Stage 2 applies six kinematic losses to FM output a_{final} only: speed, acceleration, curvature, jerk, lateral acceleration, and lateral jerk (Appendix D). Six representative polygon presets are shown in Appendix G.1.

3.4 Reward-Refined Flow Fine-tuning

Stage 3 optimizes θ to maximize:

$$J(\theta) = \mathbb{E}_{s \sim \rho, x_0 \sim \mathcal{A}}[R(s, f_\theta(x_0, s))] \quad (9)$$

where ρ is the training scene distribution and R is a black-box collision reward. Since f_θ is deterministic, $\partial a / \partial \theta$ is available via backprop; the challenge is computing $\partial R / \partial a$, which is intractable because R is non-differentiable. We estimate it via zeroth-order finite differences over the anchor table, requiring only that reward differences are *directionally informative*—a condition satisfied by collision detection. This requires fewer assumptions than FlowGRPO (which needs a Gaussian SDE approximation) and SafeDiffuser [50] (which requires differentiable CBFs). The full policy update rule is given in Eq. (13).

Step 1: Nearest Neighbor Selection. Sample anchor $k \sim \text{Uniform}(\{1, \dots, M\})$; generate query $a_i = f_\theta(x_0^{(k)}, s)$. Retrieve $\mathcal{N}(a_i)$ via Eq. (4).

Step 2: Reward Querying. Query $r_i = R(s, a_i)$ and $r_j = R(s, a_j)$ for all neighbors. Using raw anchors is valid by anchor-proximity ($a_j \approx f_\theta(a_j, s)$), saving N forward passes. The reward is defined as:

$$R(s, a) = \min\{t : \text{collision at step } t\}, \quad \text{or } T+1 \text{ if collision-free} \quad (10)$$

Setting the no-collision value to $T+1=81$ (not $T=80$) creates a one-step margin at the safety boundary, sharpening the gradient signal.

Step 3: Gradient Estimation. We assign exponentially decaying weights by distance rank:

$$w_{ij} = \exp\left(-\frac{\text{rank}(a_j | a_i)}{N-1}\right) \quad (11)$$

and aggregate reward-weighted finite differences:

$$\hat{g}_i = \frac{\sum_{j=1}^N w_{ij} \cdot \frac{r_j - r_i}{d_{ij}^2 + \epsilon_0} \cdot (a_j - a_i)}{\sum_{j=1}^N w_{ij}} \quad (12)$$

where $d_{ij}^2 = \|a_j - a_i\|_2^2$ and $\epsilon_0 > 0$ is a small numerical stability constant. Each term is a one-sided finite-difference estimate of the directional derivative of R ; the weighted sum reduces variance by aggregating reward differences across N directions.

Step 4: Policy Update. Treating $\hat{g}_i$ as a stop-gradient constant, the RL loss and combined training objective are:

$$\mathcal{L}_{\text{RL}}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \text{sg}(\hat{g}_i)^\top a_i, \quad \mathcal{L}(\theta) = \mathcal{L}_{\text{FM}}(\theta) + \lambda \mathcal{L}_{\text{RL}}(\theta) \quad (13)$$

Unlike SPSA [51] or ES [52], directions $(a_j - a_i)$ come from the FPS anchor table, guaranteeing kinematic feasibility and angular diversity across maneuver shapes.

GridMap-accelerated collision detection. To evaluate $N+1$ trajectories efficiently, we rasterize obstacles into a coarse grid and apply per-trajectory early termination; see Appendix C for details.

3.5 Training Pipeline and Inference

The three stages execute sequentially and are fully decoupled: Stage 1 trains f_θ on $\mathcal{L}_{\text{FM}}$ with the encoder pre-trained and fixed; Stage 2 jointly post-trains f_θ and EF_ϕ on $\mathcal{L}_{\text{FM}} + \mathcal{L}_{\text{EF}} + \mathcal{L}_{\text{kin}}$ (encoder frozen) and fixes ϕ after convergence; Stage 3 fine-tunes only f_θ via $\mathcal{L}_{\text{FM}} + \lambda \mathcal{L}_{\text{RL}}$ with ϕ frozen. The anchor table $\mathcal{A}$ remains fixed throughout. Algorithm 1 gives the complete procedure.

Algorithm 1 DriveAnchor Three-Stage Training Pipeline

Require: Flow matching model f_θ , Energy Field module EF_ϕ , training data $\mathcal{D}$, reward function R
Require: Anchor table $\mathcal{A} = \{a_k^*\}$ (pre-built via FPS from historical corpus), neighbors N , loss weight λ

- 1: **Stage 1 (Demonstration Flow):** Train f_θ on $\mathcal{D}$ with $\mathcal{L}_{\text{FM}}$ {anchor-conditioned FM, establishes diversity}
 - 2: **Stage 2 (Guided Flow):** Jointly train f_θ and EF_ϕ on 7.5M unified driving samples with $\mathcal{L}_{\text{FM}} + \mathcal{L}_{\text{EF}} + \mathcal{L}_{\text{kin}}$ (encoder frozen); fix ϕ after convergence { $\mathcal{L}_{\text{kin}}$ on FM output only; ϕ fixed before Stage 3}
 - 3: **Stage 3 (Reward-Refined Flow):** Starting from Stage 2 checkpoint with ϕ frozen:
 - 4: **for** each training batch $(s, x_1) \in \mathcal{D}$ **do**
 - 5: Sample anchor $k \sim \text{Uniform}(\{1, \dots, M\})$; generate $a_i = f_\theta(x_0^{(k)}, s)$ {one anchor per scene per step; full vocabulary covered stochastically across batches}
 - 6: Retrieve $\{a_j\}_{j=1}^N \leftarrow \mathcal{N}(a_i)$ from $\mathcal{A}$ {KD-tree lookup on FM output}
 - 7: Query rewards: $r_i = R(s, a_i)$; $r_j = R(s, a_j)$ for all j {GridMap-accelerated collision detection}
 - 8: Compute $\hat{g}_i$ via Eq. (12) {detach from graph}
 - 9: $\mathcal{L} \leftarrow \mathcal{L}_{\text{FM}}(\theta) - \lambda \cdot \text{sg}(\hat{g}_i)^\top a_i$ {only FM parameters updated; ϕ frozen}
 - 10: Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
 - 11: **end for**
-

Inference and top- K selection. All $M=2,398$ anchors are evaluated in a single forward pass; candidates are ranked by $\|\hat{\Delta}x^{(k)}\|_2$ where $\hat{\Delta}x^{(k)} \triangleq f_\theta(x_0^{(k)}, s) - x_0^{(k)}$ (lower = higher confidence, anchor close to GT maneuver). The top- K set is then passed to a downstream cost-based selector decoupled from training, allowing arbitrary cost functions.

4 Experiments

4.1 Setup

Dataset. Stages 1 and 3 use 120M real-world driving samples covering highway scenarios including intersections, roundabouts, lane changes, and ramp merges; Stage 2 uses a balanced 7.5M subset covering all EF polygon presets. The FPS corpus (100M+ frames, temporally disjoint from training and evaluation) builds $\mathcal{A}$. Evaluation is on $\sim 2\text{M}$ held-out scenarios with all $M=2,398$ anchors per scene, yielding an exhaustive trajectory distribution rather than a single-sample estimate.

Metrics. Trajectory accuracy: for a predicted trajectory $\hat{a}$ vs. ground truth a^* ,

$$\text{ADE} = \frac{1}{H} \sum_{t=1}^H \|\hat{a}_t - a_t^*\|_2, \quad \text{FDE} = \|\hat{a}_H - a_H^*\|_2 \quad (14)$$

$\min_{\text{ADE/FDE}}@H$ takes the minimum over all 2,398 trajectories ($H \in \{30, 80\} = 3 \text{ s}/8 \text{ s}$) and measures diversity coverage; gt_ADE/FDE uses the nearest-to-GT prediction and measures fidelity—a

model can improve min-ADE by spreading trajectories without degrading gt-ADE. **Safety:** near-range ($R < 40$) and far-range ($R < 80$) collision rates, where R is defined in Eq. (10) (fraction of trajectories colliding before step 40 or 80, respectively, aggregated across all M anchors and all test scenes). **Reward:** mean R from Eq. (10) (higher is better).

Baselines. **FM* n :** n sequential FM inference passes, where each pass feeds the previous output as anchor; FM*2 is the primary non-RL baseline as the second pass consistently reduces min-ADE over FM*1. **FMRL* n :** same with RL-fine-tuned FM. **EF*1+FM/FMRL* n :** one EF pass prepended. We report original-prior and EF-modified-prior groups separately: cross-group comparisons test whether EF adds corridor controllability while preserving safety, reward, and diversity coverage; within-group comparisons isolate RL or additional FM passes under the same prior type.

Implementation. All configurations share the same Stage 2 checkpoint. Hyperparameters: $N=16$, $\lambda=0.2$, $\epsilon_0=10^{-6}$; top- K ($K=50$) candidates passed to a downstream cost-based selector. Network architecture details are in Appendix B. At deployment, the full model runs in **2.06 ms/scene** on NVIDIA Drive Orin (float16). Training uses PyTorch 2.3, AdamW (lr= 10^{-5} , weight decay 0.01, batch 128/GPU); training compute per stage is reported in Appendix E.

4.2 Main Results

FM*2 vs. FMRL*2 (Table 1). FMRL*2 reduces near-range collisions by 89% ($27.2\% \rightarrow 2.9\%$) and far-range by 87%, while mean reward improves by 32%. GT errors are virtually unchanged (gt_ADE@30: 0.23; gt_ADE@80: $0.42 \rightarrow 0.40$), confirming safety gains preserve imitation quality. **EF-guided configurations.** Adding RL (EF*1+FMRL*1) cuts near-range collisions from 13.1% to 4.2% and improves min_ADE@80 ($2.35 \rightarrow 1.25$). Compared with FMRL*2, EF*1+FMRL*2 matches performance while adding corridor guidance (near collision: $2.9\% \rightarrow 1.9\%$; reward: $78.46 \rightarrow 78.87$; min_ADE@80: $1.34 \rightarrow 0.95$). Higher gt_ADE reflects corridor-oriented anchor re-location, not degradation. In top-50 (Table 2), FMRL*2 cuts near collision $29 \times$ ($13.7\% \rightarrow 0.47\%$).

Table 1: Results over $\sim 2M$ scenarios. EF ($\dagger$) tests corridor controllability and performance retention across groups; strict ablations are within-group; bold marks within-group best.

Metric	Original prior		EF-modified prior ($\dagger$)		
	FM*2	FMRL*2	EF*1 + FM*1	EF*1 + FMRL*1	EF*1 + FMRL*2
<i>Trajectory Accuracy</i>					
min_ADE@30 $\downarrow$	0.35	0.25	0.56	0.28	0.22
min_FDE@30 $\downarrow$	0.80	0.54	1.27	0.55	0.44
gt_ADE@30 $\downarrow$	0.23	0.23	0.32	0.31	0.29
gt_FDE@30 $\downarrow$	0.40	0.39	0.60	0.58	0.57
min_ADE@80 $\downarrow$	2.27	1.34	2.35	1.25	0.95
min_FDE@80 $\downarrow$	4.42	2.82	4.44	2.65	1.87
gt_ADE@80 $\downarrow$	0.42	0.40	0.76	0.73	0.71
gt_FDE@80 $\downarrow$	0.96	0.89	1.72	1.63	1.59
<i>Safety</i>					
Near-range collision $\downarrow$	0.2722	0.0294	0.1313	0.0423	0.0188
Far-range collision $\downarrow$	0.5654	0.0715	0.2627	0.1178	0.0873
<i>Reward</i>					
Mean reward $\uparrow$	59.22	78.46	70.13	77.14	78.87

Table 2: Top-50 trajectories (ranked by velocity magnitude).

Metric	FM*2	FMRL*2	EF*1+FM*1	EF*1+FMRL*1	EF*1+FMRL*2
Near-range $\downarrow$	0.1373	0.0047	0.2276	0.0264	0.0062
Far-range $\downarrow$	0.2780	0.0224	0.4024	0.0842	0.0441
Mean reward $\uparrow$	71.05	80.48	63.22	78.54	80.14

Qualitative visualization. Figure 2 shows FM*2 diversity, FMRL*2 safety concentration, EF corridor targeting, and their combination; EF*1+FMRL*2 is in Table 2.

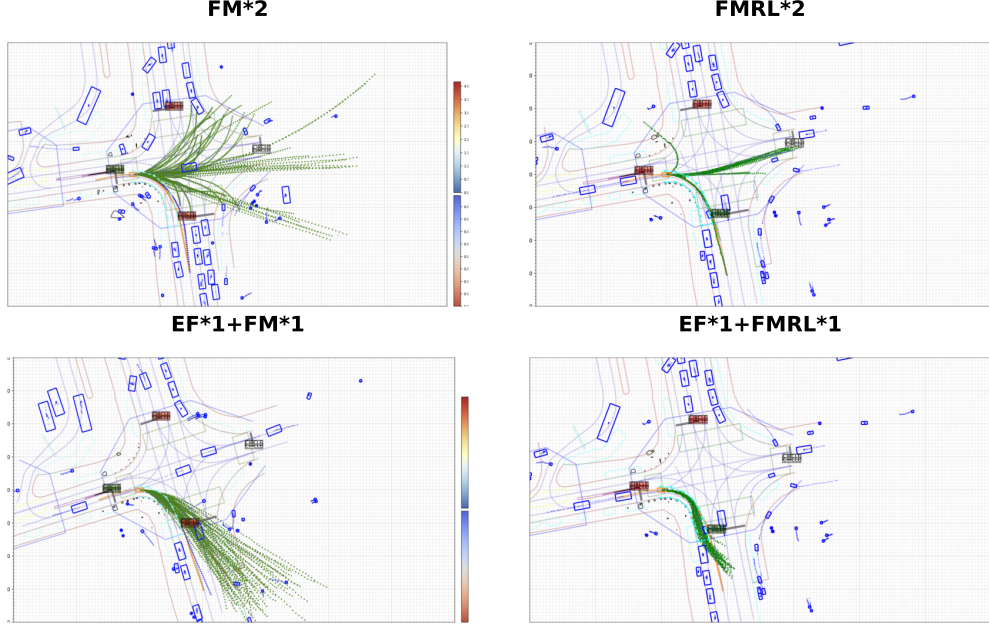


Figure 2: Top-50 trajectory distributions for four configurations.

Training dynamics. All stages converge stably (Figure 3); Stage 3 improves reward while preserving imitation quality (Appendix G.2). The FM loss remains close to the frozen baseline, indicating that reward optimization improves safety without overwriting imitation behavior.

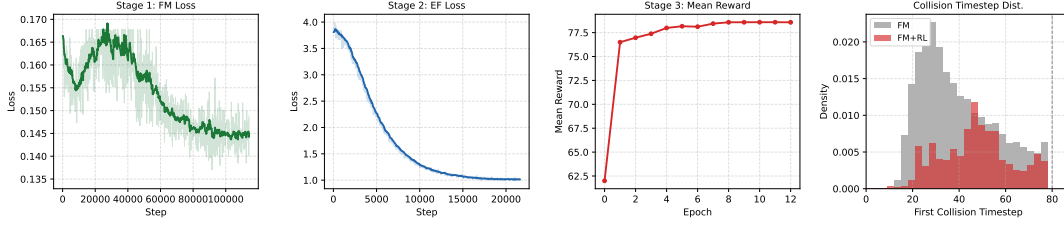


Figure 3: Training convergence and safety improvement across stages.

Real-world validation. DriveAnchor has been tested on public roads (Figure 4), including intersections, pedestrian crossings, and dense merging/lane-change traffic. These tests use the same single-step deployment graph as offline evaluation, including EF rematching before FM decoding.



Figure 4: DriveAnchor validated on a production autonomous driving platform (NVIDIA Drive Orin, float16).

4.3 Ablation Studies

We ablate the number of ZO gradient neighbors N in Stage 3 (Appendix F). $N=4$ covers only $4/160=2.5\%$ of the 160-D trajectory space and yields near-zero RL progress; $N=16$ performs best

and is used throughout. This suggests that modest directional coverage is enough for reward alignment once the anchor vocabulary already provides diverse candidate behaviors.

5 Conclusion and Limitations

DriveAnchor is a composable three-stage framework that grounds behavioral diversity in a structured anchor vocabulary, adds run-time controllability via a geometry-driven Energy Field, and aligns safety through zeroth-order RL—three fully decoupled stages each independently updatable. The FPS-constructed anchor vocabulary converts corpus coverage into deterministic generative diversity; EF injects corridor constraints before generation without differentiable guidance, requiring only EF updates for new presets; deterministic single-step inference reduces reward optimization to a finite-difference direction search in anchor space. The result is an 89% reduction in near-range collision rates and a 32% gain in mean reward, with negligible change in imitation accuracy and 2.06 ms inference on Drive Orin. Post-deployment evaluation on internal production metrics further confirms these gains, with Miles Per Intervention (MPI) and Miles Per Accident (MPA) both consistently improved over the previous production system.

Limitations. All experiments use an internal dataset without public-benchmark evaluation (nuPlan [53], NAVSIM [54], Waymo [55]). Richer reward signals beyond the current collision-based formulation are an important future direction. The FPS anchor vocabulary’s coverage of rare maneuvers is constrained by the corpus distribution. Finally, the framework omits the explicit flow-time parameter t , leaving the model’s theoretical convergence properties for future formalization.

Acknowledgments

This work was conducted at Meituan Autonomous Driving. We thank the Meituan Autonomous Driving team for their support and helpful discussions.

References

- [1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- [2] T. Tan, Y. Zheng, R. Liang, Z. Wang, K. Zheng, J. Zheng, J. Li, X. Zhan, and J. Liu. Flow matching-based autonomous driving planning with advanced interactive behavior modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 38310–38335. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/36d1e8aa9ceec3b781682bf5e63c31bf-Abstract-Conference.html.
- [3] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15, pages 627–635. PMLR, 2011. URL <https://proceedings.mlr.press/v15/ross11a.html>.
- [4] P. de Haan, D. Jayaraman, and S. Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 11698–11709, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/947018640bf36a2bb609d3557a285329-Abstract.html>.
- [5] J. Ho and T. Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021. URL <https://openreview.net/forum?id=qw8AKxfYbI>.
- [6] Z. Zhong, D. Rempe, Y. Chen, B. Ivanovic, Y. Cao, D. Xu, M. Pavone, and B. Ray. Language-guided traffic simulation via scene-level diffusion. In *Proceedings of The 7th Conference on Robot Learning (CoRL)*, volume 229, pages 144–177. PMLR, 2023. URL <https://proceedings.mlr.press/v229/zhong23a.html>.

- [7] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang. Flow-GRPO: Training flow matching models via online RL. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 40783–40818. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3a10c46572628d58cb44fb705f25cbbf-Abstract-Conference.html.
- [8] K. Black, M. Janner, Y. Du, I. Kostrikov, and S. Levine. Training diffusion models with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=YCWjhGrJFD>.
- [9] T. Zhang, C. Yu, S. Su, and Y. Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 106282–106319. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/98d6f928497d9eaad9f320fb2db3040d-Abstract-Conference.html.
- [10] Z. Xue, J. Wu, Y. Gao, F. Kong, L. Zhu, M. Chen, Z. Liu, W. Liu, Q. Guo, W. Huang, and P. Luo. DanceGRPO: Unleashing GRPO on visual generation. *arXiv preprint arXiv:2505.07818*, 2025. URL <https://arxiv.org/abs/2505.07818>.
- [11] J. Ho, A. N. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.
- [12] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=StlgiaRCHLP>.
- [13] M. Janner, Y. Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162, pages 9902–9915. PMLR, 2022. URL <https://proceedings.mlr.press/v162/janner22a.html>.
- [14] Y. Zheng, R. Liang, K. Zheng, J. Zheng, L. Mao, J. Li, W. Gu, R. Ai, S. E. Li, X. Zhan, and J. Liu. Diffusion-based planning for autonomous driving with flexible guidance. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=wM2sfVgMDH>. Oral.
- [15] C. M. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, and D. Anguelov. MotionDiffuser: Controllable multi-agent motion prediction using diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9644–9653, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Jiang_MotionDiffuser_Controllable_Multi-Agent_Motion_Prediction_Using_Diffusion_CVPR_2023_paper.html.
- [16] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Peebles_Scalable_Diffusion_Models_with_Transformers_ICCV_2023_paper.html.
- [17] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=li7qeBbCR1t>.
- [18] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=CD9Snc73AW>. Expert Certification.

- [19] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17853–17862, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Hu_Planning-Oriented_Autonomous_Driving_CVPR_2023_paper.html.
- [20] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang. VAD: Vectorized scene representation for efficient autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8340–8350, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Jiang_VAD_Vectorized_Scene_Representation_for_Efficient_Autonomous_Driving_ICCV_2023_paper.html.
- [21] A. Prakash, K. Chitta, and A. Geiger. Multi-modal fusion transformer for end-to-end autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7077–7087, 2021. URL https://openaccess.thecvf.com/content/CVPR2021/html/Prakash_Multi-Modal_Fusion_Transformer_for_End-to-End_Autonomous_Driving_CVPR_2021_paper.html.
- [22] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Proceedings of The 7th Conference on Robot Learning (CoRL)*, volume 229, pages 1268–1281. PMLR, 2023. URL <https://proceedings.mlr.press/v229/dauner23a.html>.
- [23] J. Cheng, Y. Chen, X. Mei, B. Yang, B. Li, and M. Liu. Rethinking imitation-based planners for autonomous driving. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 14123–14130, 2024. doi:10.1109/ICRA57147.2024.10611364. URL <https://ieeexplore.ieee.org/document/10611364>.
- [24] J. Gao, C. Sun, H. Zhao, Y. Shen, D. Anguelov, C. Li, and C. Schmid. VectorNet: Encoding HD maps and agent dynamics from vectorized representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11525–11533, 2020. URL https://openaccess.thecvf.com/content_CVPR_2020/html/Gao_VectorNet_Encoding_HD_Maps_and_Agent_Dynamics_From_Vectorized_Representation_CVPR_2020_paper.html.
- [25] M. Liang, B. Yang, R. Hu, Y. Chen, R. Liao, S. Feng, and R. Urtasun. Learning lane graph representations for motion forecasting. In *European Conference on Computer Vision (ECCV)*, volume 12347 of *Lecture Notes in Computer Science*, pages 541–556. Springer, 2020. doi:10.1007/978-3-030-58536-5_32. URL https://link.springer.com/chapter/10.1007/978-3-030-58536-5_32.
- [26] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. L. Rus, and S. Han. BEVFusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2774–2781, 2023. doi:10.1109/ICRA48891.2023.10160968. URL <https://ieeexplore.ieee.org/document/10160968>.
- [27] Y. Chai, B. Sapp, M. Bansal, and D. Anguelov. MultiPath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 100, pages 86–99. PMLR, 2020. URL <https://proceedings.mlr.press/v100/chai20a.html>.
- [28] H. Zhao, J. Gao, T. Lan, C. Sun, B. Sapp, B. Varadarajan, Y. Shen, Y. Shen, Y. Chai, C. Schmid, C. Li, and D. Anguelov. TNT: Target-driven trajectory prediction. In *Proceedings of the 2020 Conference on Robot Learning (CoRL)*, volume 155, pages 895–904. PMLR, 2021. URL <https://proceedings.mlr.press/v155/zhao21b.html>.

- [29] J. Gu, C. Sun, and H. Zhao. DenseTNT: End-to-end trajectory prediction from dense goal sets. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15303–15312, 2021. URL https://openaccess.thecvf.com/content/ICCV2021/html/Gu_DenseTNT_End-to-End_Trajectory_Prediction_From_Dense_Goal_Sets_ICCV_2021_paper.html.
- [30] B. Varadarajan, A. Hefny, A. Srivastava, K. S. Refaat, N. Nayakanti, A. Cornman, K. Chen, B. Douillard, C. P. Lam, D. Anguelov, and B. Sapp. MultiPath++: Efficient information fusion and trajectory aggregation for behavior prediction. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 7814–7821, 2022. doi:10.1109/ICRA46639.2022.9812107. URL <https://ieeexplore.ieee.org/document/9812107>.
- [31] N. Nayakanti, R. Al-Rfou, A. Zhou, K. Goel, K. S. Refaat, and B. Sapp. Wayformer: Motion forecasting via simple & efficient attention networks. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2980–2987, 2023. doi:10.1109/ICRA48891.2023.10160609. URL <https://ieeexplore.ieee.org/document/10160609>.
- [32] S. Shi, L. Jiang, D. Dai, and B. Schiele. Motion transformer with global intention localization and local movement refinement. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 6531–6543, 2022. URL https://papers.nips.cc/paper_files/paper/2022/hash/2ab47c960bfee4f86dfc362f26ad066a-Abstract-Conference.html.
- [33] S. Shi, L. Jiang, D. Dai, and B. Schiele. MTR++: Multi-agent motion prediction with symmetric scene modeling and guided intention querying. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 46(5):3955–3971, 2024. doi:10.1109/TPAMI.2024.3352811. URL <https://ieeexplore.ieee.org/abstract/document/10398503>.
- [34] Z. Zhou, J. Wang, Y.-H. Li, and Y.-K. Huang. Query-centric trajectory prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17863–17873, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Zhou_Query-Centric_Trajectory_Prediction_CVPR_2023_paper.html.
- [35] J. Ngiam, V. Vasudevan, B. Caine, Z. Zhang, H.-T. L. Chiang, J. Ling, R. Roelofs, A. Bewley, C. Liu, A. Venugopal, D. J. Weiss, B. Sapp, Z. Chen, and J. Shlens. Scene transformer: A unified architecture for predicting future trajectories of multiple agents. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=Wm3EA50lHsG>.
- [36] Z. Zhou, L. Ye, J. Wang, K. Wu, and K. Lu. HiVT: Hierarchical vector transformer for multi-agent motion prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8823–8833, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Zhou_HiVT_Hierarchical_Vector_Transformer_for_Multi-Agent_Motion_Prediction_CVPR_2022_paper.html.
- [37] Z. Zhong, D. Rempe, D. Xu, Y. Chen, S. Veer, T. Che, B. Ray, and M. Pavone. Guided conditional diffusion for controllable traffic simulation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3560–3566, 2023. doi:10.1109/ICRA48891.2023.10161463. URL <https://ieeexplore.ieee.org/document/10161463>.
- [38] A. Ajay, Y. Du, A. Gupta, J. B. Tenenbaum, T. S. Jaakkola, and P. Agrawal. Is conditional generative modeling all you need for decision making? In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=sP1fo2K9DFG>.
- [39] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions

- with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [40] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
 - [41] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 15084–15097, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/7f489f642a0ddb10272b5c31057f0663-Abstract.html>.
 - [42] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. doi:10.15607/RSS.2023.XIX.026. URL <https://www.roboticsproceedings.org/rss19/p026.html>.
 - [43] S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora. Fine-tuning language models with just forward passes. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 53038–53075, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a627810151be4d13f907ac898ff7e948-Abstract-Conference.html.
 - [44] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=XVjTT1nw5z>.
 - [45] R. T. Q. Chen and Y. Lipman. Flow matching on general geometries. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=g7ohD1TITL>.
 - [46] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=PxTIG12RRHS>.
 - [47] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202, pages 32211–32252. PMLR, 2023. URL <https://proceedings.mlr.press/v202/song23a.html>.
 - [48] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5099–5108, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/d8bf84be3800d12f74d8b05e9b89836f-Abstract.html>.
 - [49] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, and F. J. Huang. A tutorial on energy-based learning. In *Predicting Structured Data*. MIT Press, 2006. URL <http://yann.lecun.com/exdb/publis/pdf/lecun-06.pdf>.
 - [50] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus. SafeDiffuser: Safe planning with diffusion probabilistic models. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=ig2wk7kK9J>.
 - [51] J. C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi:10.1109/9.119632. URL <https://ieeexplore.ieee.org/document/119632>.

- [52] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017. URL <https://arxiv.org/abs/1703.03864>.
- [53] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. In *CVPR 2021 Workshop on Autonomous Driving*, 2021. URL <https://arxiv.org/abs/2106.11810>.
- [54] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta. NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 28706–28719. Curran Associates, Inc., 2024. doi:10.52202/079017-0902. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/32768f7faf1995026ef9821c696f3404-Paper-Datasets_and_Benchmarks_Track.pdf.
- [55] N. Montali, J. Lambert, P. Mougin, A. Kuefler, N. Rhinehart, M. Li, C. Gulino, T. Emrich, Z. Yang, S. Whiteson, B. White, and D. Anguelov. The waymo open sim agents challenge. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 59151–59171. Curran Associates, Inc., 2023. doi:10.52202/075280-2582. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/b96ce67b2f2d45e4ab315e13a6b5b9c5-Paper-Datasets_and_Benchmarks.pdf.

A Discussion: Design Insights and Future Directions

Why rematch after EF relocation. In the EF+FM pipeline (Eqs. (5)–(6)), EF first displaces the anchor from $a_{\text{anchor}} \in \mathcal{A}$ to a corridor-guided proposal $\tilde{a}_{\text{EF}} = a_{\text{anchor}} + \Delta v_{\text{EF}}$. Because FM is trained only on the anchor vocabulary and does not cover the whole 160-D trajectory space, the shifted proposal should be rematched to the nearest anchor before decoding. Empirically, feeding the shifted proposal directly into FM degrades kinematic quality, so we project it back to the vocabulary with $a_{\text{rematch}} = \arg \min_{a \in \mathcal{A}} \|\tilde{a}_{\text{EF}} - a\|_2$ and then decode with FM. A more principled solution is to align FM training coverage with the anchor partition, motivating the Voronoi-based sampling below.

Toward Voronoi-based anchor sampling. The rematch step works because FM’s training implicitly partitions trajectory space by proximity to $\mathcal{A}$ —an approximation to the natural Voronoi partition of $\mathcal{A}$. The current ϵ -ball filter is a hard-radius proxy for this partition: too large a radius causes gradient contamination across maneuver modes; too small leaves the flow field with uncovered regions (Figure 5, left). The principled solution is to compute the exact Voronoi diagram of $\mathcal{A}$ and draw each noisy training input from within its Voronoi cell (Figure 5, right). Under this scheme, rematch becomes exact by construction, FM*2 (two sequential passes) directly corresponds to a two-step Euler integration converging toward the exact ODE solution, and kinematic degradation after EF relocation is structurally eliminated.

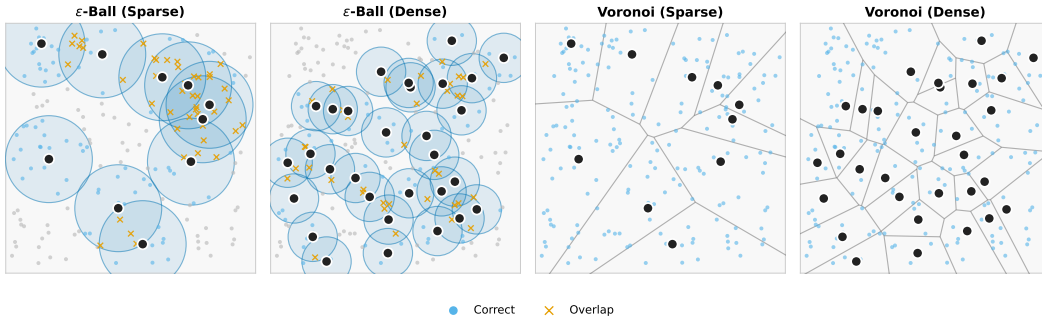


Figure 5: ϵ -ball vs. Voronoi sampling under dense and sparse anchor regimes (six simulated driving modes, colored; anchors as white-bordered circles). ϵ -ball creates overlapping regions and uncovered gaps (grey); Voronoi cells provide a complete, non-overlapping partition in both regimes.

Temporal guidance for EF. EF currently enforces only spatial constraints (polygon entry and exit edge). Introducing a time constraint—requiring the trajectory to reach a specified waypoint within T' seconds—would enable temporal guidance, providing richer behavioral control and finer temporal coordination for timing-critical scenarios such as gap selection during merging. Concretely, a temporal EF could encode a time-budget window alongside the spatial polygon, so that the planner must jointly satisfy both positional and scheduling requirements. This is particularly valuable when spatial feasibility alone is insufficient: for instance, entering a roundabout before an oncoming vehicle closes the gap, or passing an intersection within a traffic-light green phase. The EF specification would then become a compact spatial-temporal contract, and the decoder would need to internalize not only *where* but also *when* the trajectory should pass through key regions.

B Network Architecture

Table 3 details the architecture of each module. The scene encoder is pre-trained on an internal base planning model and frozen throughout Stages 2 and 3; only the FM and EF decoders are trained. The full model output per scene is $[B, 2, 398, 160]$ (80×2 displacement vectors over all anchors).

Table 3: Network architecture and training hyperparameters for reproducibility.

Module	Parameter	Value
<i>Shared Encoder (frozen in Stages 2–3)</i>		
	Input streams	ego, moving/static obstacles, road graph, traffic light
	Transformer layers	6
	Token dim (d_{model})	384
	Attention heads	6
	Output projection	MLP(384 $\rightarrow$ 256)
	Output shape	$[B, 634, 256]$
<i>Anchor Vocabulary & Shared Projector</i>		
	Vocabulary size M	2,398 trajectories
	Anchor dim	$80 \times 2 = 160$
	Trajectory projector (shared)	MLP(160 $\rightarrow$ 256); shared by FM and EF
	MLP block	Linear $\rightarrow$ LayerNorm $\rightarrow$ GELU $\rightarrow$ Linear $\rightarrow$ Dropout
<i>FM Decoder (Stages 1 & 3 trainable)</i>		
	Cross-attention heads	2
	d_{model}	256
	Velocity head	MLP(256 $\rightarrow$ 256) $\rightarrow$ MLP(256 $\rightarrow$ 160)
	FM supervision	SmoothL1, target = GT $- \tilde{x}_0$
	Inference passes	2 (FM*2)
	Output	$[B, K, 160]$ top- K trajectories ($K=50$)
<i>EF Decoder (Stage 2 trainable)</i>		
	EF input	33-dim: 1 scene type + 16×2 polygon coords
	Scene projector	MLP(1 $\rightarrow$ 128)
	Polygon projector	MLP(32 $\rightarrow$ 128)
	Fusion projector	concat(128, 128, 256) $\rightarrow$ MLP(512 $\rightarrow$ 256)
	Cross-attention heads	2
	d_{model}	256
	Energy head	MLP(256 $\rightarrow$ 256) $\rightarrow$ MLP(256 $\rightarrow$ 160)
	EF supervision	good: target = 0; bad: target = $a_{\text{near-good}} - a_{\text{noisy}}$
<i>Training Hyperparameters</i>		
	Optimizer	AdamW, lr = 10^{-5} , batch 128/GPU
	FM / EF loss weight	0.005
	RL loss weight λ	0.2
	ZO neighbors N	16
	ϵ -ball threshold	per-cluster median inter-anchor distance
	Kinematic constraints	speed, accel, jerk, curvature, lat-accel, lat-jerk

C GridMap-Accelerated Collision Detection

Naively checking all $N+1$ trajectory–obstacle pairs scales as $\mathcal{O}((N+1) \cdot T \cdot |\mathcal{O}|)$. We use a two-stage strategy: (1) **Build**: rasterize all obstacles (road boundaries and static objects) into a coarse 2D grid (10 m/cell) via Bresenham traversal; each cell stores overlapping obstacle IDs. (2) **Query**: project each ego OBB onto the grid to retrieve 2–4 candidate obstacles, then apply precise OBB intersection. Per-trajectory early termination skips remaining waypoints once a collision is detected. This substantially reduces per-scene evaluation while preserving exact collision semantics.

D Stage 2 EF Post-training: Kinematic Constraint Configuration

Two loss forms are used for the six constraints. For upper-bound constraints (speed, acceleration, curvature, lateral acceleration), we use a sigmoid soft penalty that provides a smooth, non-zero gradient just past the boundary (in contrast to a hard hinge):

$$\mathcal{L}_{\text{max}}(x, x_{\text{max}}) = \mathbb{E}[\text{ReLU}(\sigma(x - x_{\text{max}}) - 0.5)] \quad (15)$$

where σ is the sigmoid function. For smoothness constraints (jerk, lateral jerk), we use a ReLU-squared penalty that strongly penalizes large violations:

$$\mathcal{L}_{\text{smooth}}(x, x_{\text{max}}) = \mathbb{E}[\max(0, |x| - x_{\text{max}})^2] \quad (16)$$

Table 4 details the six losses, their ego-history context, threshold constants, and final loss weights after scaling.

Table 4: Kinematic constraint loss configuration for Stage 2 EF post-training. “History” indicates how many ego-history frames are concatenated as input; “threshold” is the per-constraint safety bound; “weight” is the product of the base weight and the loss scale coefficient. Per-step penalty formulas are listed separately in Table 5.

Loss	Constraint	Ego history	Threshold	Effective weight
speed_limitation_loss	Speed upper bound	1 frame	$v_{\max} = 25.0 \text{ m/s}$	$1.0 \times 0.1 = 0.1$
acceleration_limitation_loss	Acceleration upper bound	2 frames	$a_{\max} = 4.5 \text{ m/s}^2$	$0.01 \times 0.01 = 1 \times 10^{-4}$
jerk_limitation_loss	Jerk smoothness	3 frames	$j_{\max} = 4.5 \text{ m/s}^3$	$0.001 \times 0.1 = 1 \times 10^{-4}$
curvature_limitation_loss	Curvature upper bound	2 frames	$\kappa_{\max} = 0.4339 \text{ m}^{-1}$	$0.005 \times 0.1 = 5 \times 10^{-4}$
lateral_acceleration_limitation_loss	Lateral acceleration bound	2 frames	$a_{\text{lat},\max} = 1.0 \text{ m/s}^2$	$0.01 \times 0.005 = 5 \times 10^{-5}$
lateral_jerk_limitation_loss	Lateral jerk smoothness	3 frames	$j_{\text{lat},\max} = 1.0 \text{ m/s}^3$	$0.001 \times 0.1 = 1 \times 10^{-4}$

Table 5: Per-step penalty formula for each kinematic constraint loss. Upper-bound constraints use the sigmoid soft penalty from Eq. (15); smoothness constraints use the ReLU-squared penalty from Eq. (16).

Loss	Formula
speed_limitation_loss	$\text{ReLU}(\sigma(v - v_{\max}) - 0.5)$
acceleration_limitation_loss	$\text{ReLU}(\sigma(a - a_{\max}) - 0.5)$
curvature_limitation_loss	$\text{ReLU}(\sigma(\kappa - \kappa_{\max}) - 0.5)$
lateral_acceleration_limitation_loss	$\text{ReLU}(\sigma(a_{\text{lat}} - a_{\text{lat},\max}) - 0.5)$
jerk_limitation_loss	$\max(0, j - j_{\max})^2$
lateral_jerk_limitation_loss	$\max(0, j_{\text{lat}} - j_{\text{lat},\max})^2$

E Training Compute

All three stages are trained on A100-80G GPUs using PyTorch 2.3 with AdamW (lr= 10^{-5} , weight decay 0.01, batch size 128/GPU). Stage 1 pretrains the FM backbone over 120M real-world samples; Stage 2 jointly trains EF and FM over a 7.5M balanced subset; Stage 3 runs zeroth-order RL fine-tuning with 64 GPUs for a short duration. Table 6 summarises the wall-clock time and GPU-hour budget for each stage.

Table 6: Training compute per stage.

Stage	GPUs	Wall time	GPU-Hours
Stage 1 (FM Pretraining)	32×A100-80G	28.2 h	902
Stage 2 (EF Post-training)	32×A100-80G	16.9 h	541
Stage 3 (RL Fine-tuning)	64×A100-80G	3.8 h	243

F Ablation Studies

Table 7: Ablation on N (anchor neighbors). $N=4$ is the reference row; arrows show improvement vs. $N=4$.

N	Near-range	Far-range	Mean reward
4	0.2752	0.5675	59.10
8	0.0304↓	0.0769↓	78.25↑
16	0.0294 ↓	0.0715 ↓	78.46 ↑

Effect of anchor neighbors N . As shown in Table 7, $N=4$ provides only 4/160=2.5% directional coverage in 160-D trajectory space, yielding near-zero RL progress; $N=8$ recovers most of the gain and $N=16$ adds a marginal further improvement.

G Additional Figures

G.1 EF Polygon Presets

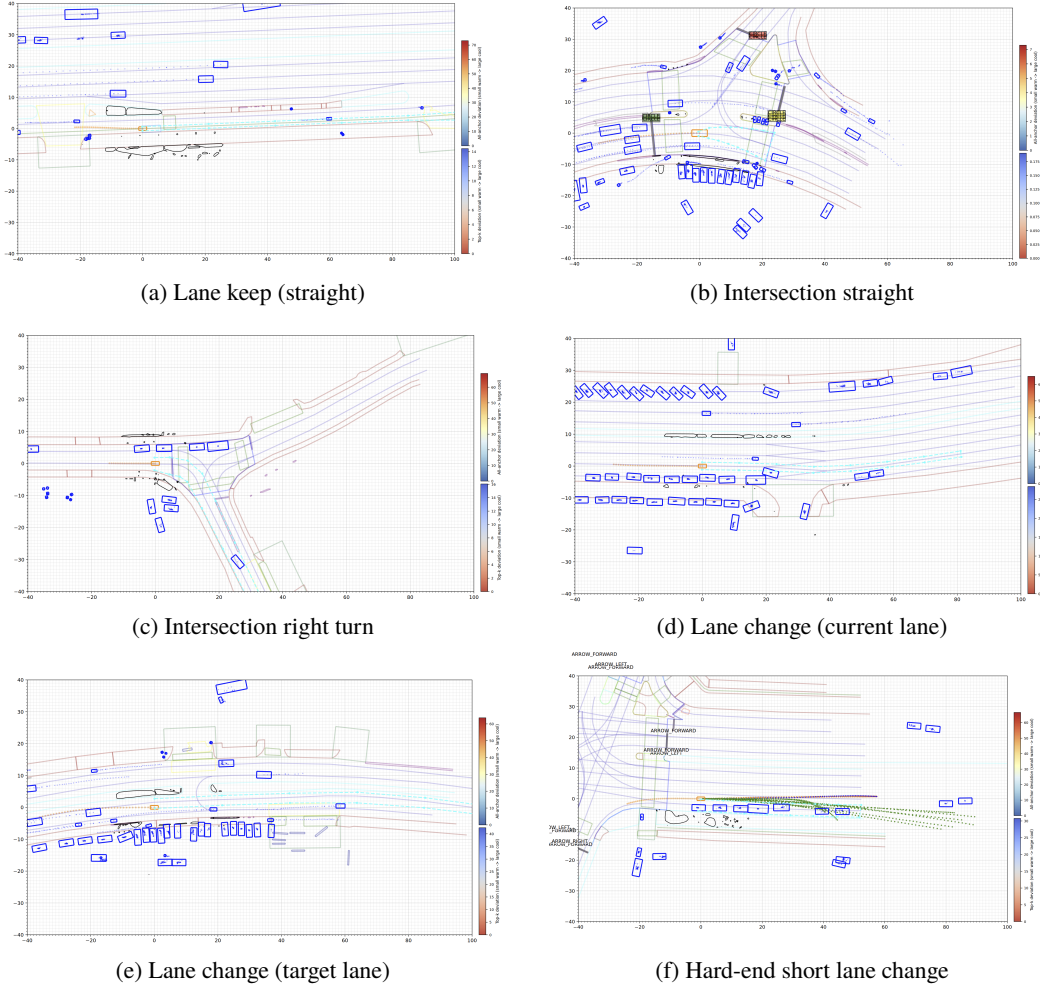


Figure 6: EF polygon presets for six driving scenarios. Each polygon specifies where the trajectory must enter and exit; the designated exit edge encodes the target maneuver. The hard-end short lane-change preset (bottom right) constrains the change to complete within a short spatial window.

G.2 Training and Visualization

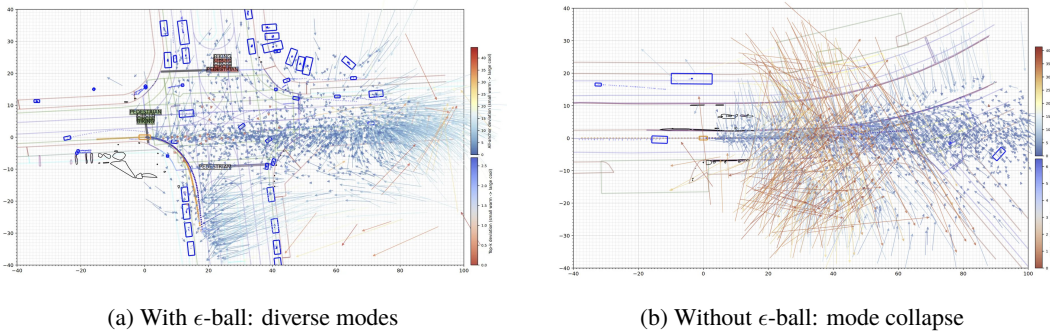


Figure 7: Stage 1 diversity vs. mode collapse. The anchor-conditioned FM generates diverse trajectory modes (**left**); without the ϵ -ball constraint, sparse anchor regions cause cross-mode contamination and convergence to a single dominant mode (**right**).

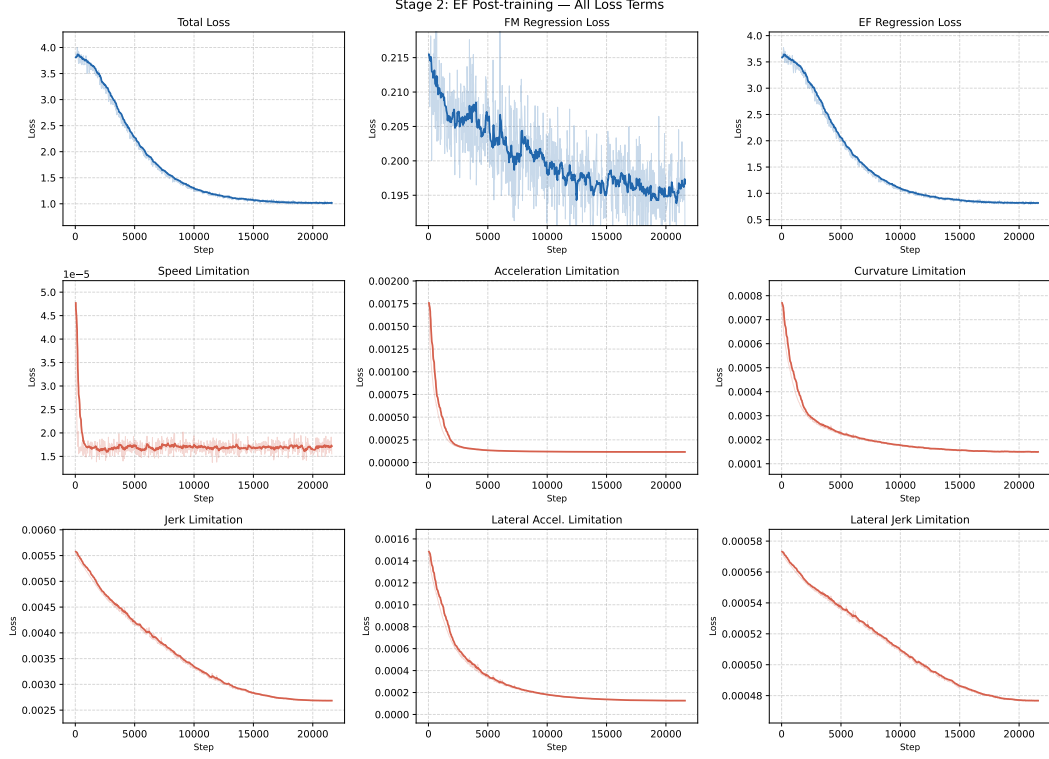


Figure 8: Stage 2 EF Post-training: all loss terms over 21.6K steps on 7.5M unified driving samples. **Top row** (blue): total loss, FM regression loss, EF regression loss. **Bottom two rows** (red): six kinematic constraint losses penalizing violations of speed, acceleration, curvature, jerk, lateral acceleration, and lateral jerk limits.

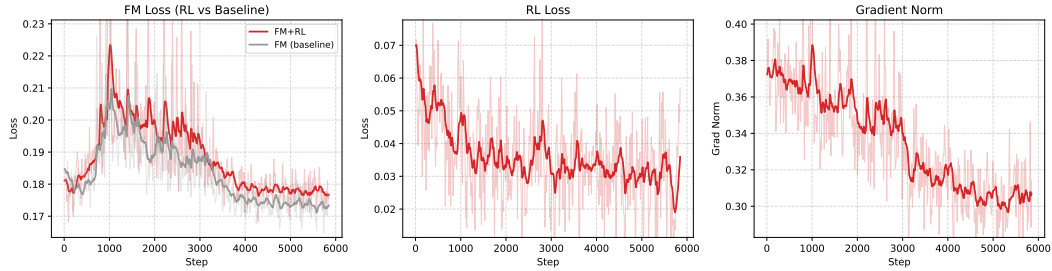


Figure 9: Stage 3 RL Fine-tuning training dynamics over 5.8K steps. **Left**: FM loss of the RL-fine-tuned model (red) versus the frozen SFT baseline (gray)—both remain comparable, confirming that RL fine-tuning does not degrade imitation quality. **Center**: RL loss ($\mathcal{L}_{RL}$), which reflects the zeroth-order reward gradient signal. **Right**: gradient norm, showing stable optimization throughout fine-tuning. Note: Stage 3 does not include kinematic constraint losses; the reward signal is a pure black-box collision-detection score.